



Software Designing Techniques

Program development cycle

1. **Analyze:** Define the problem.
2. **Design:** Plan the solution to the problem.
3. **Code:** Translate the algorithm into a programming language.
4. **Test and debug:** Locate and remove any errors in the program.
5. **Deployment and Maintenance:** Getting software out and continue to make improvements and changes.

Software Design Phase

- Software design is a process to conceptualize the software requirements into software implementation.
- Software design takes the user requirements as challenges and tries to find optimum solution.
- While the software is being conceptualized, a plan is chalked out to find the best possible design for implementing the intended solution.

Designing Tools

- There are several designing tools that are used to help in converting proposed solution into computer programs:
- **Flowchart** - Graphically depicts the logical steps to carry out a task and shows how the steps relate to each other.
- **Pseudocode** - Uses English-like phrases with some basic programming terms to outline the program.
- **Finite and Extended state machine**
- **Data Flow Diagrams**
- **Entity Relation Diagrams**

Flowchart

- A Flow Chart (also known as a Process Flow Diagram or Process Map) is a diagram of the steps in a process and their sequence.
- There are two types of flow charts diagrams:
 - A **high-level flowchart**, outlining 6-10 major steps, gives a high-level view of a process. These flowcharts display the major blocks of activity, or the major system components, in a process.
 - A **detailed flowchart** is a close-up view of the process, typically showing dozens of steps. These flowcharts make it easy to identify complexity, excessive steps, etc. in a process and should be used when you want to standardize or make changes in the process.

When to Use a Flow Chart?

- When you need to define or analyze an existing process.
- When you need to standardize or redesign a process.
- When you need to find areas for improvement in a process such as unnecessary steps, gaps, barriers, etc.

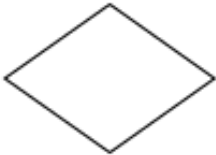
How Flow Charts Are Constructed:

1. Identify the goal for creating the flowchart and the level of detail required-high or detailed.
2. Assemble the people who know the process best and outline the process steps.
3. Define the first and last steps in the process.
4. Begin documenting the process steps in sequence. Some steps may be parallel-they happen at the same time. Describe the process as it really exists, not the ideal.
5. Work through the entire process, showing actions and decisions appropriately in the order they occur. Link these together using arrows to show the flow of the process.

How Flow Charts Are Constructed:

6. At decision symbols, choose the most natural branch and continue to the end.
7. Use notes for unfamiliar steps and continue to the end.
8. When you reach the last step, go back to fill in any branches.
9. Follow up on unfamiliar steps and update chart.
10. Validate your flow chart. Work from step to step asking yourself and others if you have correctly represented the sequence of actions and decisions involved in the process.
11. Identify areas for improvement and redesign the process.

Flowchart symbols

Symbol	Name	Meaning
	<i>Flowline</i>	Used to connect symbols and indicate the flow of logic.
	<i>Terminal</i>	Used to represent the beginning (Start) or the end (End) of a task.
	<i>Input/Output</i>	Used for input and output operations, such as reading and displaying. The data to be read or displayed are described inside.
	<i>Processing</i>	Used for arithmetic and data-manipulation operations. The instructions are listed inside the symbol.
	<i>Decision</i>	Used for any logic or comparison operations. Unlike the input/output and processing symbols, which have one entry and one exit flowline, the decision symbol has one entry and two exit paths. The path chosen depends on whether the answer to a question is "yes" or "no."

Flowchart symbols continued



Connector

Used to join different flowlines.



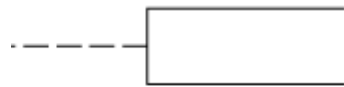
Offpage Connector

Used to indicate that the flowchart continues to a second page.



Predefined Process

Used to represent a group of statements that perform one processing task.



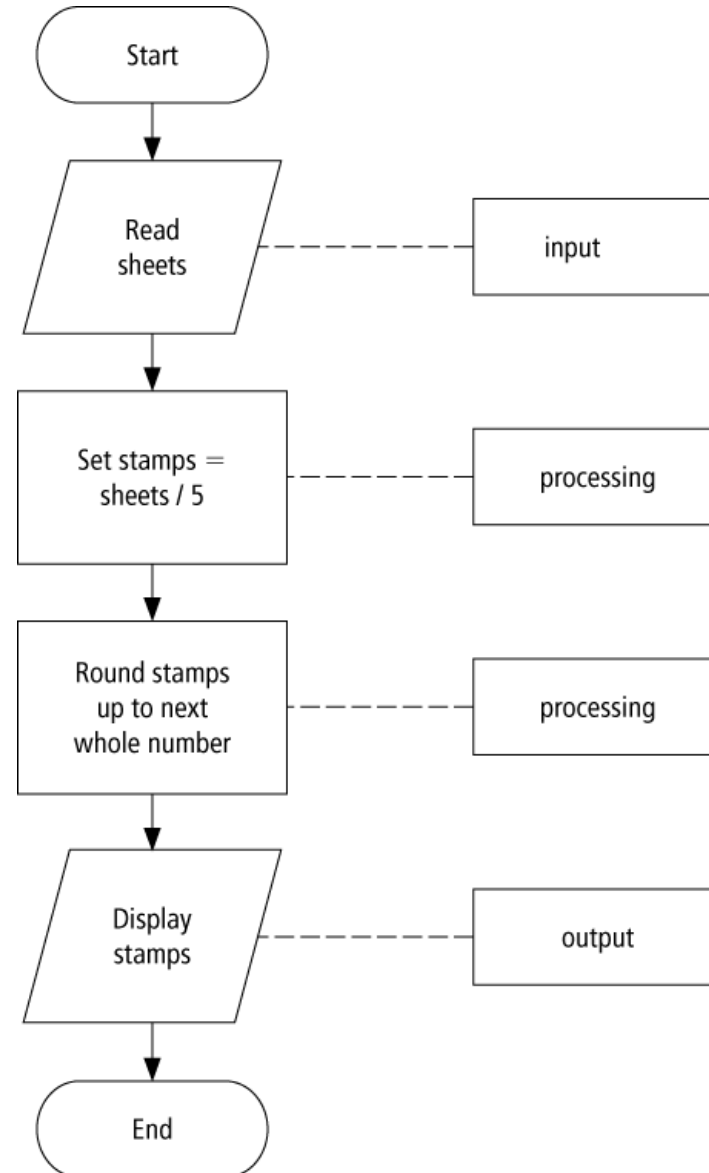
Annotation

Used to provide additional information about another flowchart symbol.

Problem solving example

- How many stamps do you use when mailing a letter?
- One rule of thumb is to use one stamp for every five sheets of paper or fraction thereof.
- How to solve this problem?

Flowchart example



Pseudocode

- Pseudocode is a method of describing computer algorithms using a combination of natural language and programming language.
- It is essentially an intermittent step towards the development of the actual code. It allows the programmer to formulate their thoughts on the organization and sequence of a computer algorithm without the need for actually following the exact coding syntax.
- Although pseudocode is frequently used there are no set of rules for its exact implementation.

Pseudocode example

Determine the proper number of stamps for a letter

Read Sheets (*input*)

Set the number of stamps to $\text{Sheets} / 5$ (*processing*)

Round the number of stamps up to the next whole number (*processing*)

Display the number of stamps (*output*)

Finite State Machine (FSM)

- A state machine diagram models the behavior of a single object, specifying the sequence of events that an object goes through during its lifetime in response to events.
- It is an abstract machine that can be in exactly one of a finite number of states at any given time.
- Type of state machine:
 - **Finite state machine** (FSM): the transition is associated with a set of input Boolean conditions and a set of output Boolean functions.
 - **Extended finite state machine** (EFSM) model, the transition can be expressed by an “if statement” consisting of a set of trigger conditions. If trigger conditions are all satisfied, the transition is fired, bringing the machine from the current state to the next state

Door Finite State Machine

- As an example, the following state machine diagram shows the states that a door goes through during its lifetime.

1

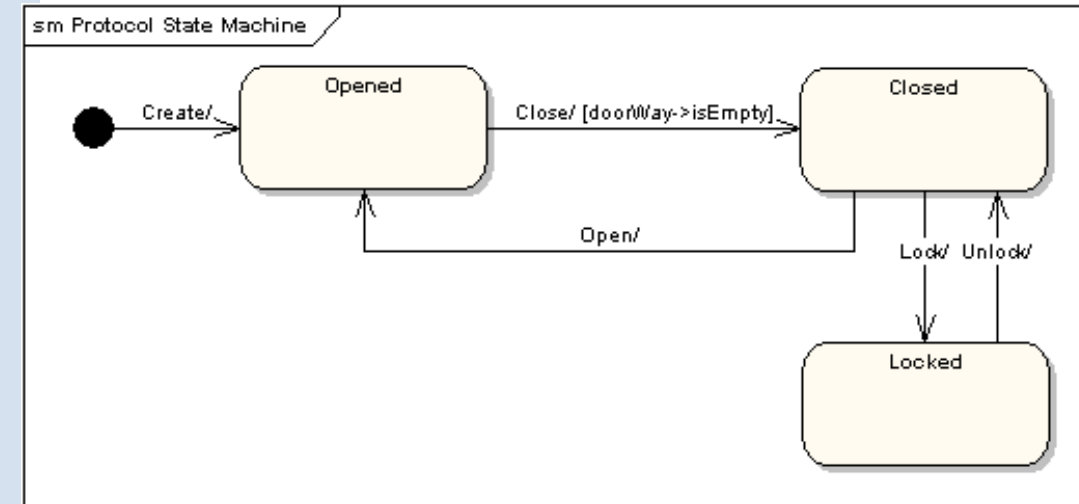
The door can be in one of three states: "Opened", "Closed" or "Locked". It can respond to the events Open, Close, Lock and Unlock.

2

Notice that not all events are valid in all states; for example, if a door is opened, you cannot lock it until you close it.

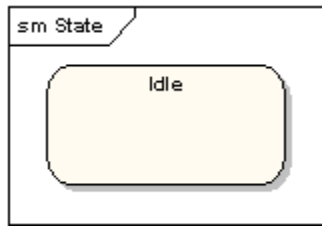
3

Also notice that a state transition can have a guard condition attached: if the door is Opened, it can only respond to the Close event if the condition `doorWay->isEmpty` is fulfilled.



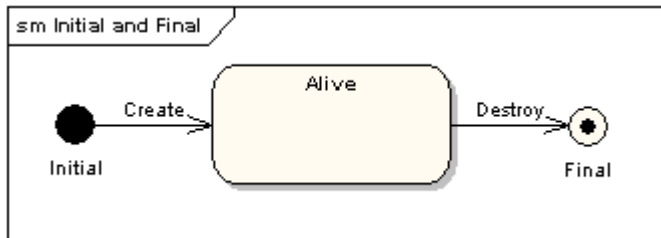
State Machine symbols

States



A state is denoted by a round-cornered rectangle with the name of the state written inside it.

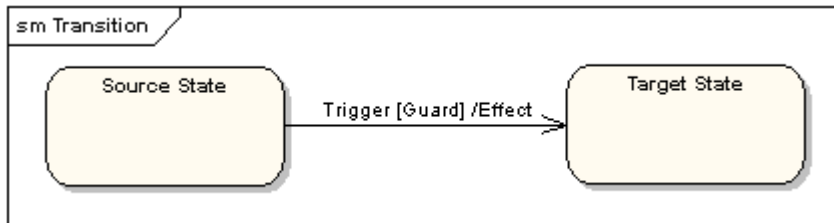
Initial and Final States



The initial state is denoted by a filled black circle and may be labeled with a name. The final state is denoted by a circle with a dot inside and may also be labeled with a name.

- "Guard" is a condition which must be true in order for the trigger to cause the transition.
- "Effect" is an action which will be invoked directly on the object that owns the state machine as a result of the transition.

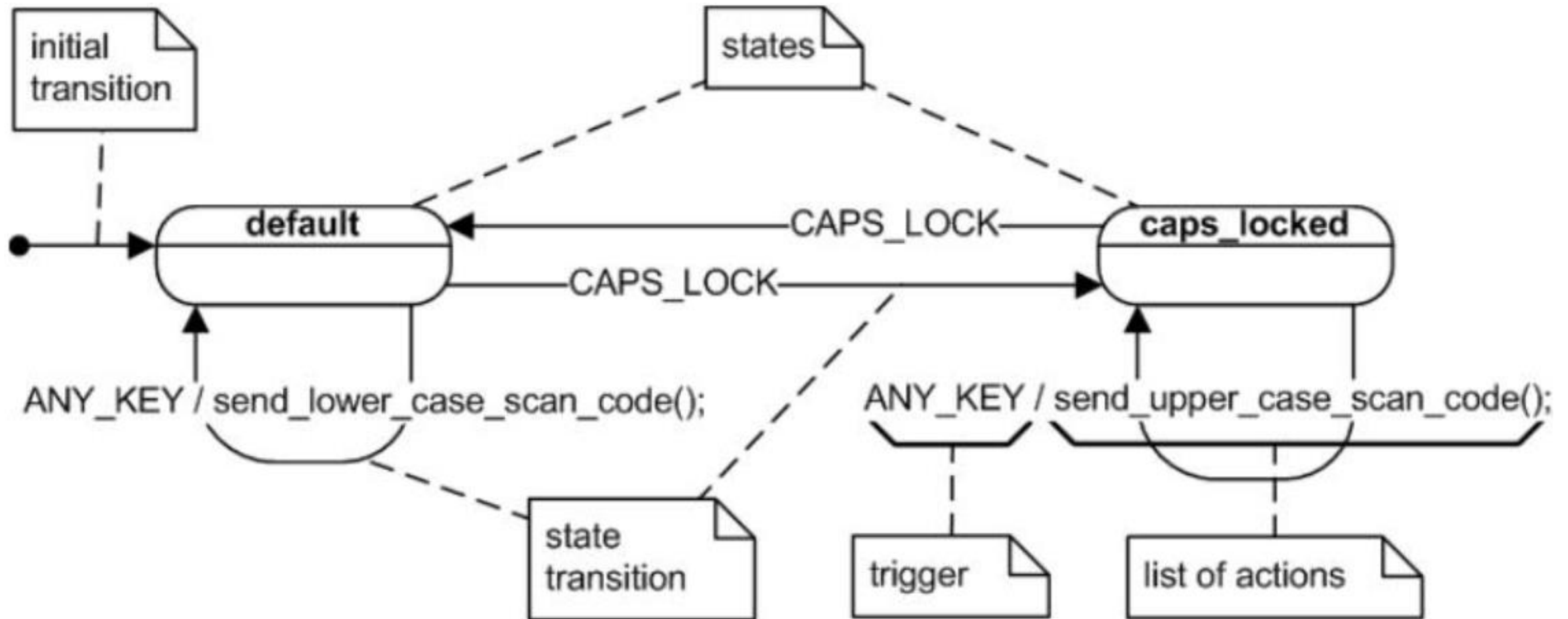
Transitions



Transitions from one state to the next are denoted by lines with arrowheads. A transition may have a trigger, a guard and an effect, as below.

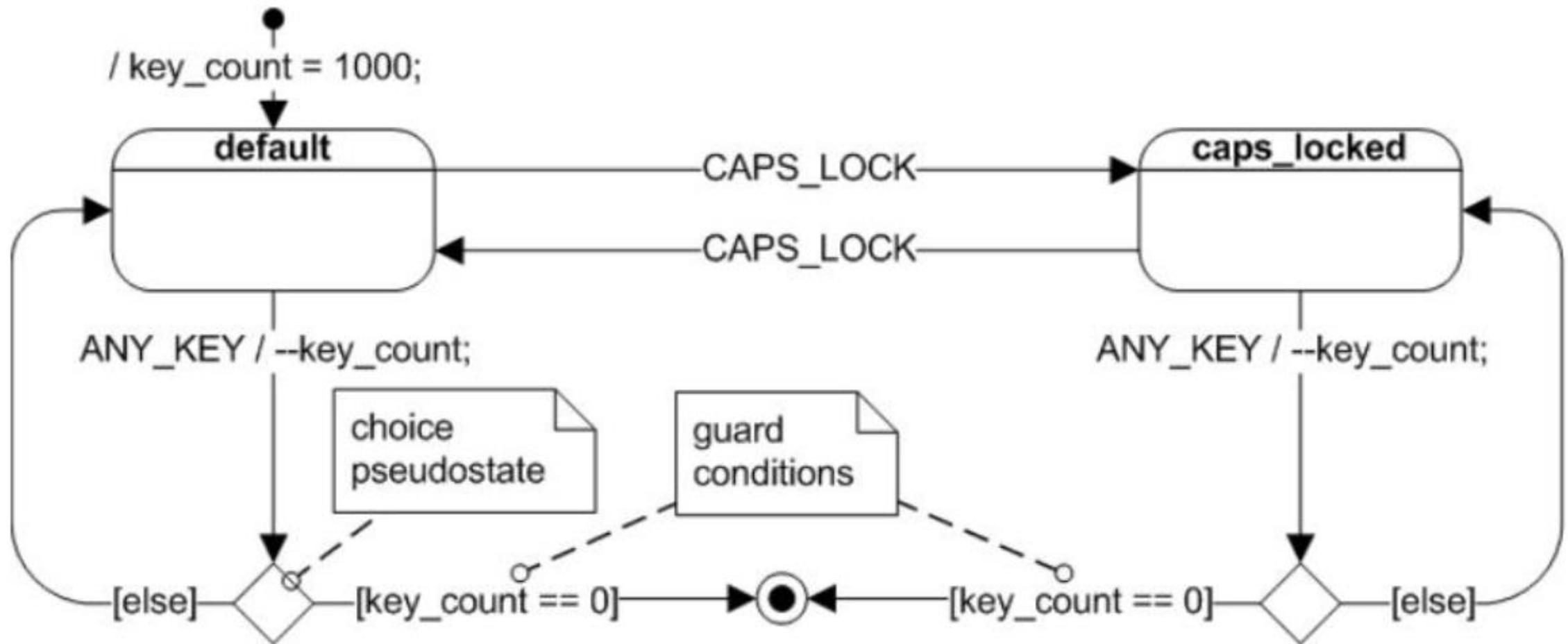
Finite State Machine Example

Representing the computer keyboard state machine



Extended Finite State Machine Example

“cheap keyboard” with extended state variable `key_count` and various guard conditions.



Data Flow Diagrams (DFDs)

- Data flow diagram is a graphical representation of the flow of data through an information system.
- It is one of the most commonly used systems-modeling tools, particularly in the operational system in which functions of the system are the most important and more complex than the data that the system manipulates.
- Rather than showing the strict order of execution of steps:
 - It shows how processes depend on one another for information.
 - It shows data flow from external into the system and shows how the data moved from one process to another process.
- System analysts use it as a tool for modeling and analyzing the processes in the system. It helps analysts to visualize the data processes since data enter to the system, and then they are used by the system until they are returned to the environment.
- Analysts use DFDs to study alternative information handling procedures during the process of designing new information services.
- Data Flow Diagrams can be used for compared the new system and the old system. With this comparison, system analysts can find the gap between two systems and the effectiveness of the improved system.

Data Flow Diagram Components

- **Squares or Ovals** represent external entities, terminators, sources, or sinks. Typically, a terminator is a person or a group of people outside the control of the system being modeled. It represents where information comes from and where it goes. There are three important things that we must remember about terminators:
 - They must be outside the system we are being modeled.
 - As a result, neither the system analysts nor system designers are able to change the contents, organization and the internal process within the terminators.
 - Any relationship between each terminator is not a part of the system we are considering, so the system analysts and designers can not include any relationship that exists between terminators
- **Circles or Rounded Rectangles** represent processes within the system. It shows a part of the system that transforms inputs into outputs. The name of the process in the symbols usually explains what the process does so that it is generally used with verb-object phrase.
- **Arrows** represent the data flows. It can be either be electronic data or physical items or both. The name of the arrows represents the meaning of the packet (data or items) that flow along.
- **Open-ended Rectangles** represent data stores, including both electronic stores and physical stores. Data stores might be used for accumulating data for a long or short period of times.

Data Flow Diagram Levels



In the real projects, data flow diagrams are considerably large and complex. So, we need to avoid drawing the whole system in only one diagram.



System analysts organize the overall DFD in a series of levels so that each level provides successively more detail about a portion of the level above it.



The top-level DFD is called a **context diagram** which represents the system as a single process. It consists of only one circle representing the entire system, the data flows showing the interaction between the system and the external terminators.



There is no data stores appear on a context diagram because the data stores of the systems are conceptually inside the one process.

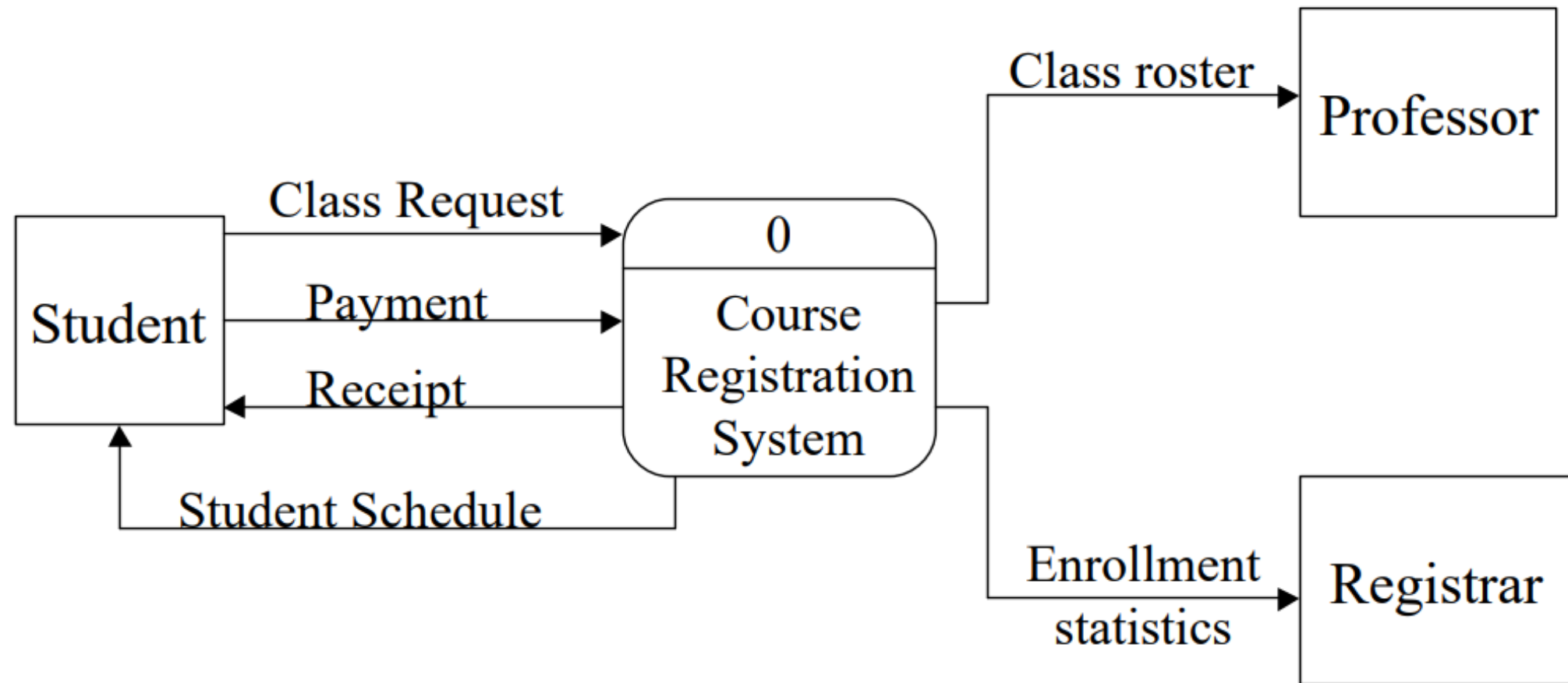


The next level DFD are called Level-0 DFD, Level-1 DFD, and so on.

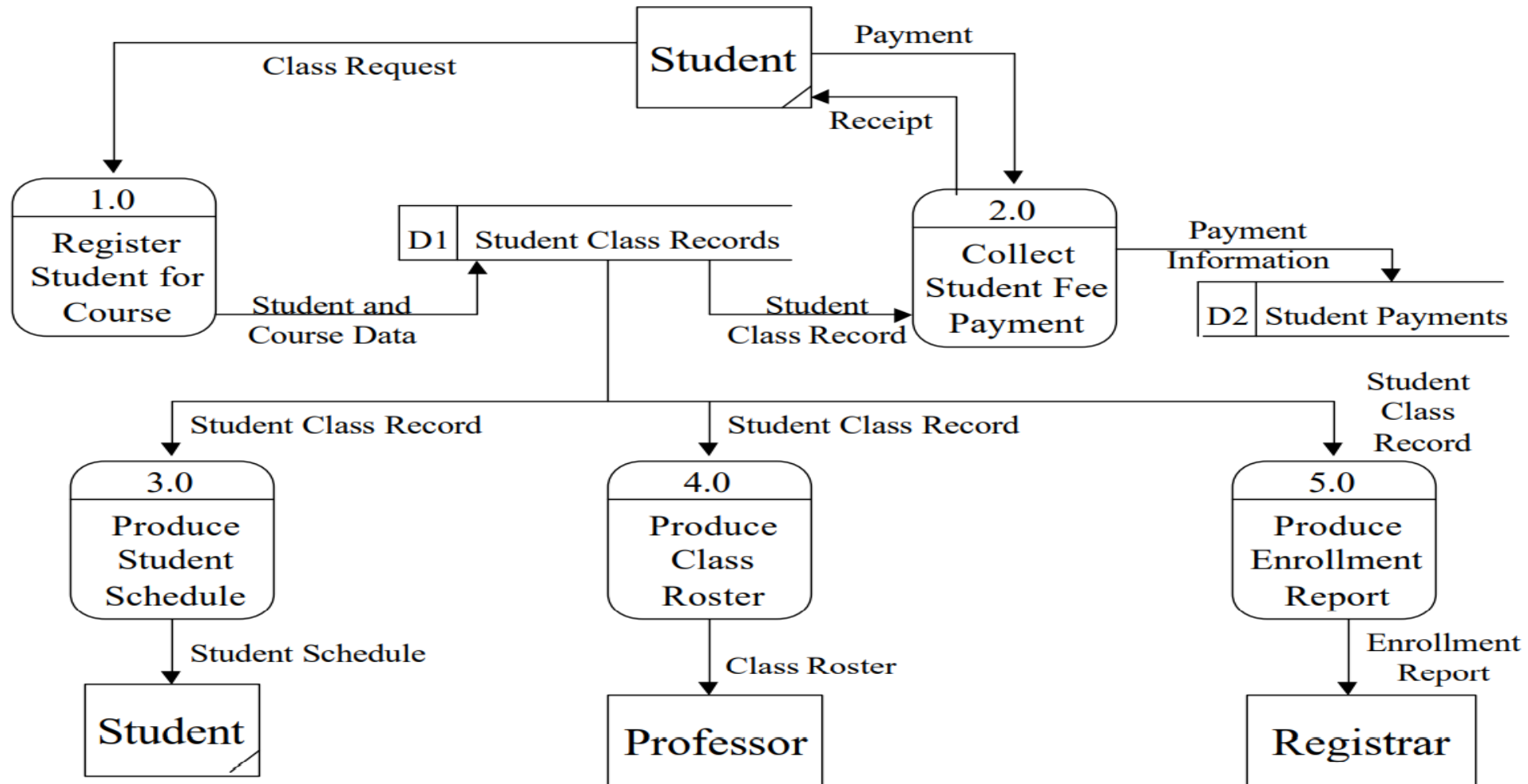


Level-0 diagram is the decomposition of the one process from the context diagram into **two to nine high-level processes**. Then, each process in the Level-0 diagram can be decomposed into the next level.

Course Registration: Context level Diagram



Course Registration: Current Logical Level 0 Diagram



Entity-Relationship Diagrams (ERDs)

- Entity-Relationship Diagrams is a network model that describes the stored data layout of a system at a high level of abstraction.
- Entity-Relationship Diagram concentrates on the relationships between data stores on the Data Flow Diagram that can be seen only in the process specification.
- Since Data Flow Diagram mainly focuses on the functionality, using Entity-Relationship Diagram is important to capture another part of system analysis which concentrates mainly on the data in the system.

Entity-Relationship Components

- **Entities** represent set of objects in the real world. Entities are represented by rectangles. It has three main characteristics:
 1. Each entity can be identified uniquely in some way.
 2. Each entity must play a necessary role in the system. In other words, objects which have no role in the system can not put into the system.
 3. Each entity can be described by one or more data elements. For instance, Textbook can be described by combined attributes (Title, Author, Published Date, and Edition).
- **Relationships** represent set of connections between entities. It captures how two or more entities are related to one another.
-
- **Associative object type indicator** represents something that functions both as an object and a relationship.
- **Subtype/Supertype indicators** represent objects and one or more subcategories connected by a relationship. For example, Student can be connected with Part-time Student and Full-time Student via unnamed relationship. So, in this case, Student is Supertype, and Part-time Student and Full-time Student are subtypes.

ERD Example

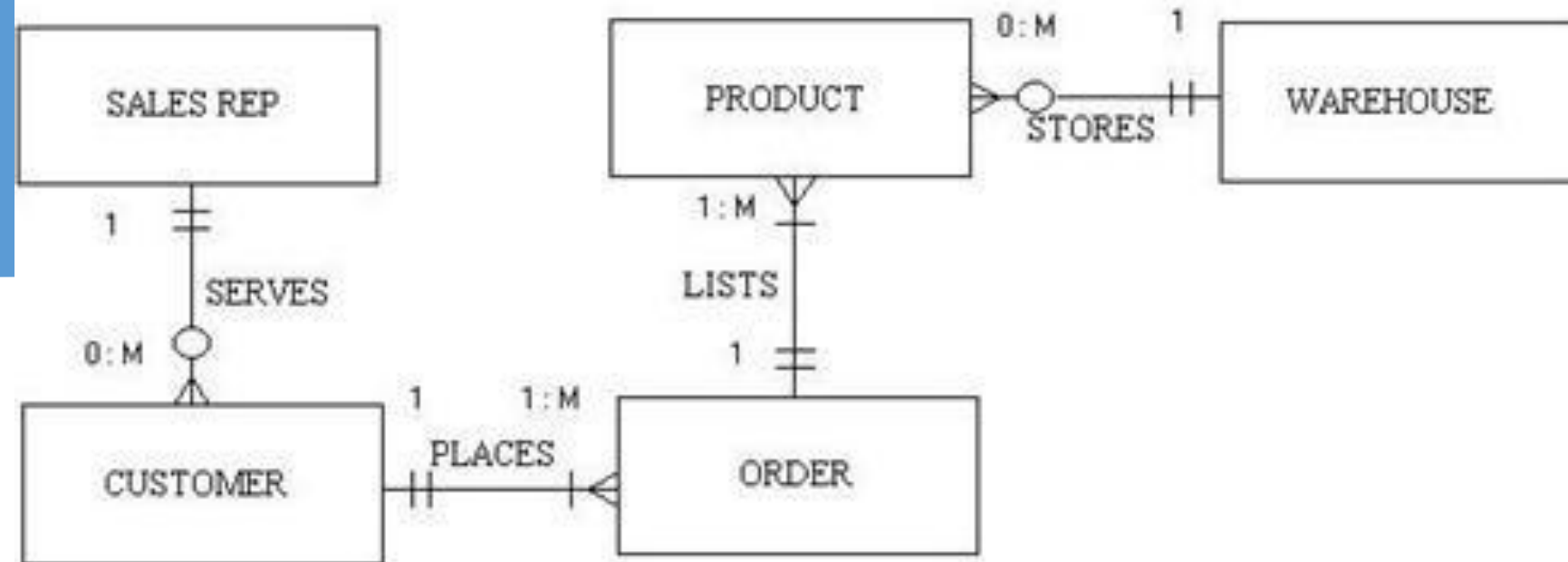


Figure 1. Entity-Relationship Diagram

- * 1 INSTANCE OF A SALES REP SERVES 1 TO MANY CUSTOMERS
- * 1 INSTANCE OF A CUSTOMER PLACES 1 TO MANY ORDERS
- * 1 INSTANCE OF AN ORDER LISTS 1 TO MANY PRODUCTS
- * 1 INSTANCE OF A WAREHOUSE STORES 0 TO MANY PRODUCTS

Refs

- An Introduction to Computers and Problem Solving: Chapter 1
- <https://www.med.unc.edu/neurosurgery/wp-content/uploads/sites/460/2018/10/Flow-chart-Process-Flow.pdf>
- [PPT - Finite State Machine PowerPoint Presentation, free download - ID:2648505 \(slideserve.com\)](#)
- [Table of Contents for 2. FSM, HSM, and Statecharts \(fsu.edu\)](#)
- [State Machine Diagram - UML 2 Tutorial | Sparx Systems](#)
- [Comparison of Diagramming Tools \(umsl.edu\)](#)
- [lect0913.PDF \(umbc.edu\)](#)